

Attempt 4 Literature Review

What we actually tried and why it stopped

Corrected next-meeting draft — not yet presented

2026-07-22

1. First, Correct the Main Misunderstanding

The original Attempt 4 was **not** a progressive-pruning proposal.

Original Attempt 4

Use the full fixed-width word once. Allow arbitrary scalar level counts and encode them with a mixed-radix address.

Later prefix successor

Read a prefix for a coarse pass, then read the full word for a more accurate pass.

Correction

These are different research questions with different stop reasons.

2. Original Intuition: Use the Word More Fully

Two scalar coordinates share one four-bit word, so there are 16 possible addresses.

ordinary	2 bits + 2 bits	$4 \times 4 = 16$ states
ordinary	1 bit + 3 bits	$2 \times 8 = 16$ states
Attempt 4	arbitrary K_1, K_2	$K_1 K_2 \leq 16$

Ordinary allocation forces each level count to be a power of two. Attempt 4 also permits choices such as $(K_1, K_2) = (3, 5)$, which uses 15 addresses.

Intuition

Three useful levels on one coordinate and five on another may fit the data better than every legal power-of-two allocation.

3. Running Example: What $(3, 5)$ Proves

coordinate 1: $\{-1, 0, 1\}$

coordinate 2: $\{-2, -1, 0, 1, 2\}$

Allocation	Joint states	Error per vector
arbitrary scalar product $(3, 5)$	15	0
best power-of-two product $(4, 4)$	16	0.1
unrestricted 16-center two-dimensional codebook	16	0

Claim ceiling

The example proves that the power-of-two restriction can waste choices. It proves no natural-data, Recall, or speed advantage. The unrestricted block codebook also fits exactly.

4. How the Original Code Would Work

For labels z_1, z_2 , store one mixed-radix address:

$$u = z_1 + K_1 * z_2$$

For $(K_1, K_2) = (3, 5)$, addresses 0–14 are used and address 15 is unused. For each query, build one group table:

$$T[u] = \text{distance}(\text{query group}, \text{reconstruction } u)$$

The scan reads one fixed word and performs one lookup per group.

Not part of the mechanism

No prefix, progressive decoding, or first-stage candidate pruning.

5. What Prior Work Already Covers

Muresan–Effros (2002): optimal one-dimensional quantizers with an integer number of levels, including non-power-of-two counts. **Brandt (2010)**: learned scalar quantizers, integer bit allocation, packed database words, query tables, and nearest-neighbor scan. **Finite Scalar Quantization (2024)**: product codebooks built from small scalar level sets, including non-power-of-two factors. **Mixed-radix indexing**: the standard mapping from several finite alphabets to one integer address.

Novelty boundary

Arbitrary level counts and mixed-radix packing alone are not enough. The remaining claim must be a measured database accuracy–cost advantage.

6. BAPQ and Quicker ADC: Close, but Different

Adaptive Bit Allocation Product Quantization (BAPQ, 2016) gives different integer bit counts to independent subspaces. Five bits still means exactly 32 codewords, not an arbitrary count such as 27. **Quicker ADC (2021)** studies four-, five-, and six-bit product subcodes, packed layouts, split lookup tables, and processor cost. Its sizes also remain powers of two.

Why they matter

They do not implement $(3, 5)$ mixed radix, but they are mandatory strong baselines for allocation, packing, lookup, and scan speed.

BAPQ = adaptive bit allocation product quantization; ADC = query-to-code distance lookup.

7. Dense Rates and Arbitrary Levels Are Crowded

- **FSQ (2024)** already uses products of small scalar level sets.
- **Q-Palette (2025)** studies fractional-rate quantizers and mixed allocation.
- **FibQuant (2026)** studies dense fixed-rate random-access vector coding and exposes the advantage of a vector code over a scalar-product restriction for its canonical source.

These papers do not collectively contain the exact database scan design.

Paper-level burden

“More rate choices” is not a contribution by itself. Attempt 4 must move a measured database-systems trade-off.

FSQ = finite scalar quantization.

8. Product Quantization and Block VQ Are Two Controls

Control	Question answered
optimized product quantization	Does an ordinary power-of-two product code already obtain the same accuracy and scan speed?
same-capacity two-dimensional block vector quantization	Does the scalar-product restriction lose too much by forbidding general two-dimensional centers?

The block codebook is more expressive. Attempt 4 needs a measured reason to retain factorization: for example, lower training cost, less model state, or cheaper table construction. VQ = vector quantization.

9. What We Actually Established

- A4-0 The synthetic (3, 5) example passed. This validated the instrument only.
- A4-1S The exact frozen construction stopped on its cost gate before natural-data work.
- V2 No valid terminal scientific record was produced; runtime remains unknown.
- A4-R0 The representation cannot be inserted as an unchanged SAQ encoder swap.

Still unmeasured

Natural-data prevalence, Recall, throughput, optimized product quantization, and trained block vector quantization.

10. What the 24-Hour Stop Means

Recorded work	CPU time
exact scalar construction	4.64 hours
canonical evidence serialization	5.03 hours
completed 61-of-128-coordinate prefix	9.668 hours
registered full-pipeline lower bound	24.17 hours

The frozen ceiling was 24.0

CPU-hours, so that registered procedure stopped.

Correct interpretation

This is a cost result for one exact solver-and-evidence pipeline. It is not a lower bound for every possible implementation of arbitrary-cardinality quantization.

11. Why It Was Not an Unchanged SAQ Swap

Attempt 4 needs learned centers, per-group cardinalities, mixed-radix labels, unused-address handling, new distance tables, and a joint-label query consumer. Existing SAQ stores per-coordinate bitplanes and reconstructs its estimate using a uniform step size and a per-vector rescaling factor.

R0 decision

NO_GO_UNCHANGED_SAQ_COMPATIBILITY: equal payload bytes do not give the bytes the same meaning. Integration requires a new representation and query path.

This does not establish that arbitrary cardinalities are ineffective.

12. The Prefix Proposal Came Later

Only after R0 did the project define a broader successor:

first p bits	all B bits
coarse lookup and filtering	accurate lookup

It added a learned label permutation, nested prefix partition, and two query stages. Its protocol explicitly called itself **a new direction**.

Why progressive papers appear

They evaluate this later successor, not the original mixed-radix intuition.

13. Why Only the Prefix Successor Was Closed

- **Riskin et al. (1994)** add progressive reconstructions after fixing a fine vector quantizer.
- **Derived Codebooks (2019)** derive a coarse grouping from a fine product code, then scan and refine.
- **Polysemous Codes (2016)** learn labels for cheap filtering while retaining accurate product-code lookup.

Combining these mechanisms with a known arbitrary-level product code left no new constraint or algorithm.

S0 decision

NO_GO_DIRECT_COMPOSITION closes the prefix successor. It is not the stop reason for original Attempt 4.

14. Keep the Two Conclusions Separate

Direction	What stopped it	What remains unknown
original arbitrary-cardinality Attempt 4	frozen exact-pipeline cost; incompatible with unchanged SAQ	natural-data gain, Recall, speed, fair product/block comparison
later prefix-code successor	direct composition of known mechanisms	implementation not justified

Accurate summary

The original coding intuition remains experimentally unresolved. A later prefix reformulation was separately rejected on prior-work grounds.

15. If We Reconsider Original Attempt 4

A fresh decision should return to the original question and require:

- ① a cheaper construction method with controlled approximation error;
- ② separately optimized power-of-two product and same-capacity block-vector controls;
- ③ complete accounting of model bytes, table construction, encoding time, and scan cost; and
- ④ a frozen test of the Recall–throughput frontier.

Current boundary

This is the unresolved evidence list, not execution authorization. It does not reopen the stopped branch.

References and Decision Records I

- Muresan and Effros. "Quantization as Histogram Segmentation." DCC 2002.
- Brandt. "Transform Coding for Fast Approximate Nearest Neighbor Search in High Dimensions." CVPR 2010.
- Mentzer et al. "Finite Scalar Quantization." ICLR 2024. <https://arxiv.org/abs/2309.15505>
- Ge et al. "Optimized Product Quantization." CVPR 2013.
- Guo et al. "Adaptive Bit Allocation Product Quantization." Neurocomputing, 2016. <https://doi.org/10.1016/j.neucom.2015.07.062>
- Andre et al. "Quicker ADC." TPAMI 2021. <https://arxiv.org/abs/1812.09162>
- Lee and Song. "Q-Palette." 2025. <https://arxiv.org/abs/2509.20214>
- Lee and Kim. "FibQuant." 2026. <https://arxiv.org/abs/2605.11478>
- Riskin et al. "Index Assignment for Progressive Transmission of Full Search Vector Quantization." IEEE TIP, 1994. <https://doi.org/10.1109/83.287025>
- Andre et al. "Derived Codebooks for High-Accuracy Nearest Neighbor Search." 2019. <https://arxiv.org/abs/1905.06900>
- Douze et al. "Polysemous Codes." ECCV 2016. <https://arxiv.org/abs/1609.01882>
- Original formulation: saq_attempt4_arbitrary_cardinality_related_work_and_gate_2026_07_13.md.
- R0: saq-a4-r0-static-gate@617ad25. Prefix successor S0: saq-a4-prefix-novelty-gate@7f4c001.